

ExoHabitAI

Theory & Code Explanation

Exoplanet Habitability Prediction System

1. Project Overview

ExoHabitAI is a full-stack machine learning web application that predicts whether an exoplanet is potentially habitable using a trained XGBoost classifier. The model was trained on 33,799 exoplanet records from the NASA Exoplanet Archive and classifies planets into two categories — Habitable (Class 1) and Not Habitable (Class 0).

Metric	Value
Total Planets in Dataset	33,799
Habitable Planets (Class 1)	9,328 — 27.6%
Not Habitable (Class 0)	24,471 — 72.4%
High Confidence Habitable (>90%)	9,209
Total Model Features	304
Spectral Type OHE Columns	291
Best Model Selected	XGBoost Classifier

2. Data Preprocessing (preprocess.ipynb)

2.1 Feature Selection

From the raw NASA Exoplanet Archive dataset, 10 key features were selected that represent the most scientifically significant planetary and stellar properties:

- `pl_rade` — Planet radius in Earth radii
- `pl_bmasse` — Planet mass in Earth masses
- `pl_orbper` — Orbital period in days
- `pl_orbsmax` — Semi-major axis in Astronomical Units (AU)
- `pl_eqt` — Equilibrium temperature in Kelvin
- `pl_dens` — Planet density in g/cm³
- `st_teff` — Host star effective temperature in Kelvin
- `st_lum` — Host star luminosity in solar units
- `st_met` — Host star metallicity [Fe/H]
- `st_spectype` — Host star spectral type (string value e.g. G2 V, K5, M3 V)

2.2 Data Cleaning

The raw dataset required significant cleaning before it could be used for training. The following steps were applied in sequence:

- Duplicate rows were identified and removed using `drop_duplicates()`
- Rows where all values were missing were dropped using `dropna(how='all')`
- Missing numeric values were filled using column median — a safe method that avoids skewing the distribution
- Missing spectral type values were filled using the column mode (most frequent value)
- Invalid values were filtered out — negative radius, mass, orbital period, and zero temperature rows were removed
- Extreme outliers were clipped at the 1st and 99th percentile using `numpy.clip()` to prevent the model from overfitting to extreme values

2.3 Feature Engineering

Four additional derived features were created to help the model better capture habitability signals:

- `temp_score` — Measures how close the planet's equilibrium temperature is to Earth's (288 K reference). Formula: $1 - \text{abs}(\text{pl_eqt} - 288) / 288$
- `stellar_temp_score` — Measures how similar the host star's temperature is to our Sun (5778 K). Formula: $1 - \text{abs}(\text{st_teff} - 5778) / 5778$
- `stellar_compatibility_index` — Weighted combination of stellar scores. Formula: $0.6 \times \text{stellar_temp_score} + 0.4 \times \text{stellar_lum_score}$
- `orbital_stability_factor` — Ratio of semi-major axis to orbital period, capturing orbital dynamics. Formula: $\text{pl_orbsmax} / \text{pl_orbper}$

2.4 One-Hot Encoding & Scaling

The `st_spectype` column contains string values like 'G2 V', 'K5', 'M3 V'. Since machine learning models cannot process strings, it was converted to numeric columns using One-Hot Encoding (OHE) with `drop_first=True` to avoid multicollinearity. This produced 291 new binary columns — one for each unique spectral type. This is the main reason the model has 304 total features (9 base + 4 engineered + 291 OHE).

`StandardScaler` was then applied to all numeric columns to normalize them to zero mean and unit variance. This ensures no single feature dominates due to its scale. The leakage features (`habitability_index`, `radius_score`, `temp_score`, `distance_score`) were removed before final training as they directly encode the target variable.

3. Model Training (Model_training_1_.ipynb)

3.1 Models Compared

Three classification models were trained and evaluated to identify the best performer:

Model	Description	Evaluation
Logistic Regression	Linear baseline model, <code>max_iter=1000</code>	Fast but limited on non-linear patterns
Random Forest	200 decision trees, <code>random_state=42</code>	Good feature importance, solid F1
XGBoost	Gradient boosted trees,	Best overall F1, ROC-AUC, and

	eval_metric=logloss	accuracy
--	---------------------	----------

3.2 XGBoost Hyperparameter Tuning

RandomizedSearchCV was used with 10 random parameter combinations and 3-fold cross-validation, optimising for F1-score to handle the class imbalance (27.6% positive class). The parameter grid searched was:

```
n_estimators:    [100, 200, 300]
max_depth:       [3, 5, 7, 9]
learning_rate:   [0.01, 0.05, 0.1, 0.2]
subsample:       [0.8, 1.0]
colsample_bytree: [0.8, 1.0]
```

The final best model was saved using `joblib.dump(xgb_model, 'models/xgboost.pkl')`. The ranked predictions were saved as `habitability_ranked.csv` with the `habitability_probability` column for each planet.

4. Backend Code — app.py & utils.py

4.1 app.py — Flask REST API

app.py initialises the Flask application and defines all API endpoints. It also serves the frontend HTML files from the /frontend directory using `send_from_directory()`. The CORS extension allows the browser frontend to make API calls without cross-origin errors. At startup, the model is loaded into memory so the first prediction request is fast.

Endpoint	Method	Description
/api/predict	POST	Single planet habitability prediction — returns probability and verdict
/api/predict/batch	POST	Batch prediction for up to 500 planets — returns summary stats
/api/rank?n=10	GET	Top N most habitable planets from the ranked dataset
/api/sample	GET	Returns 12 curated sample planets for the UI auto-fill feature
/api/sample/random	GET	Returns one random planet row from the dataset
/api/health	GET	Health check — returns model status and feature count

4.2 utils.py — The Core Preprocessing Engine

utils.py solves the key challenge of the project: the user inputs only 10 values, but the XGBoost model expects exactly 304 features in a specific order. The `build_feature_vector()` function handles this automatically through the following steps:

- Step 1 — Accept 9 scaled numeric inputs and 1 spectral type string from the user
- Step 2 — Compute 4 engineered features: `stellar_temp_score`, `stellar_lum_score`, `stellar_compatibility_index`, and `orbital_stability_factor` using the same formulas as training
- Step 3 — Call `model.get_booster().feature_names` to get the exact list of 304 feature names in training order
- Step 4 — Build 291 OHE columns all set to 0, then find the matching spectral type column (e.g. `st_spectype_G0 V`) and set it to 1
- Step 5 — Create a 1-row pandas DataFrame with all 304 columns in correct order
- Step 6 — Pass to `model.predict()` and `model.predict_proba()` to get the binary prediction and probability percentage

The `predict()` function returns a clean JSON response with four fields: `habitable` (0 or 1), `probability` (0.0 to 100.0), `label` (Potentially Habitable or Not Habitable), and `confidence_band` (High above 75%, Moderate 45-75%, Low below 45%).

4.3 Input Validation

All incoming API requests are validated before being processed. The `validate_input()` function checks that all 10 required fields are present, that the 9 numeric fields can be converted to float without errors, and that `st_spectype` is a non-empty string. For batch requests, `validate_batch()` applies these checks

to every row and returns the row number with any error message. Invalid requests return HTTP 422 with a clear error description.

5. Frontend Code — `index.html`, `style.css`, `script.js`

5.1 `index.html` — Three-Page Structure

The frontend is a single HTML file with three scrollable sections. The Home section contains a hero with animated stats, a dataset badge, and three glassmorphism info cards explaining the model architecture, performance metrics, and key features. The Predictor section contains the full input form with planetary and stellar parameter fields, a sample picker panel, batch upload drop zone, and a result panel with SVG arc and metric tiles. The Docs section shows a vertical milestone timeline and a tech stack grid.

5.2 `style.css` — Space Glassmorphism Theme

The visual design uses CSS custom properties (variables) for consistent theming. Glass cards use `background: rgba` with `backdrop-filter: blur(24px)` for the frosted glass effect. The animated starfield is a fixed canvas element layered behind all content using `z-index`. The SVG probability arc uses a `linearGradient` from blue to green with animated `stroke-dasharray` to fill the circle proportionally to the habitability score.

5.3 `script.js` — API Integration

All API communication uses the Fetch API with `async/await`. The `FIELDS` array defines all form input IDs. The `UI_ONLY_FIELDS` set contains `planet_name` which is stripped from the API payload before sending — it is only used for display in the result card. The `validateForm()` function checks all numeric fields and the spectral type before allowing submission. Batch file processing supports both CSV (custom parser) and JSON formats with results downloadable as CSV.